

RepoScribe: Architecture-Aware Repository Documentation Generation with Compact Open-Source LLMs

Tuan-Anh Pham*

Faculty of Electrical and Electronic Engineering, Phenikaa School of Engineering, Phenikaa University, Hanoi, Vietnam
Email: ptattmpxd01012004@gmail.com

† SSA Lab

Instructor: Vu Hoang Dieu

Abstract—Repository-level documentation plays an important role in helping developers understand large-scale software systems. However, automatically generating holistic documentation for entire repositories remains challenging due to complex architectural dependencies, cross-module interactions, and long-context reasoning requirements.

In this work, we investigate repository-level documentation generation using compact open-source large language models in a fully offline setting. To improve structural reasoning over large codebases, we propose an architecture-aware documentation pipeline that incorporates repository graph construction, hierarchical decomposition, architecture intermediate representations, section-aware retrieval, and hierarchical summarization.

We evaluate the proposed system on the CodeWikiBench dataset using four instruction-tuned open-source models: CodeLlama-7B-Instruct, DeepSeek-Coder-6.7B-Instruct, Mistral-7B-Instruct-v0.3, and Qwen2.5-Coder-7B-Instruct. Experimental results show that the architecture-aware pipeline consistently improves documentation quality compared with the baseline generation pipeline across multiple evaluation metrics, including coverage score, structural fidelity, and key-term coverage.

I. INTRODUCTION

Comprehensive documentation is essential for understanding, maintaining, and evolving large-scale software systems. While substantial progress has been made in function-level summarization and API documentation [1], [2], generating holistic documentation at the repository level remains significantly more challenging. Repository-level documentation requires capturing architectural structure, cross-module dependencies, configuration behavior, and system-level execution flow—elements that frequently span thousands of files and exceed the reasoning scope of conventional automated tools.

Recent work by Hoang et al. [3] introduced *CodeWiki*, a unified framework for repository-level documentation generation, together with *CodeWikiBench*, a benchmark covering 22 large-scale open-source repositories and hierarchical evaluation rubrics. Their approach demonstrates strong performance using GPT-4-class proprietary models for both generation and evaluation. However, reliance on closed APIs raises important concerns regarding cost, accessibility, deployment in restricted

environments, and long-term reproducibility due to evolving model versions.

These limitations motivate a fundamental research question:

Can compact open-source Large Language Models (LLMs) reliably generate repository-level documentation in a fully offline and reproducible setting?

This question is particularly relevant for academic research and enterprise environments where API-based inference may be infeasible or undesirable. Moreover, repository-level documentation presents unique long-context challenges: large codebases frequently exceed standard context windows, requiring models to reason over structured summaries rather than isolated functions [?], [?]. Understanding how compact models behave under these constraints is crucial for assessing trade-offs between model scale, computational cost, and documentation fidelity.

In this work, we conduct a systematic empirical study of four compact 7B-scale instruction-tuned open-source models—CodeLlama-7B-Instruct [4], DeepSeek-Coder-6.7B-Instruct [5], Mistral-7B-Instruct-v0.3 [6], and Qwen2.5-Coder-7B-Instruct [7]—on the publicly available Hugging Face release of CodeWikiBench [3], [8].

Unlike prior work that relies on proprietary large-scale systems such as GPT-4 [9], we evaluate end-to-end repository-level documentation generation in a fully offline high-performance computing (HPC) environment to ensure experimental reproducibility and deployment feasibility.

Our findings indicate that compact 7B-scale models can retain moderate structural information when generating repository documentation under long-context constraints. While performance remains below GPT-4-class systems [9], the fully offline open-source setting offers substantial advantages in cost-efficiency, transparency, and reproducibility. These results highlight a promising direction for scalable repository-level documentation research while exposing remaining limitations in global architectural reasoning.

A. Holistic Documentation Generation for Large-Scale Codebases

Automatic documentation generation has been widely studied in software engineering, primarily focusing on function-level or class-level summarization [1], [2]. However, such approaches often fail to capture repository-level structure, architectural intent, and cross-module dependencies in large-scale systems.

Hoang et al. [3] introduced *CodeWiki*, a framework for holistic documentation synthesis at the repository level. *CodeWiki* integrates repository summarization with a structured benchmark, *CodeWikiBench* [3], which contains 22 large open-source repositories and hierarchical evaluation rubrics covering architectural design, module interaction, configuration behavior, and system-level functionality.

While *CodeWiki* demonstrates strong end-to-end performance using GPT-4-class proprietary models, reliance on closed APIs raises concerns regarding cost, accessibility, and reproducibility due to evolving model versions. These limitations motivate the investigation of whether compact open-source models can generate high-quality repository-level documentation in a fully offline setting.

B. Repository-Level Documentation Benchmarks

Evaluating documentation generation at the repository level remains a challenging task due to the complexity and scale of modern software systems. Traditional automatic metrics such as BLEU or ROUGE primarily capture lexical similarity and often fail to reflect structural understanding or architectural coherence.

CodeWikiBench [3] addresses this limitation by introducing hierarchical evaluation rubrics that assess documentation quality across multiple dimensions, including architectural design, module interactions, configuration behavior, and system-level functionality. This benchmark enables systematic comparison of documentation generation systems on large-scale repositories while preserving the structural complexity of real-world software projects.

C. Compact Code-Oriented Open-Source Models

Recent advances in open-source LLMs have significantly improved the capabilities of compact models. Code-specialized systems such as *CodeLlama* [4], *DeepSeek-Coder* [5], and *Qwen-Coder* [7] are pretrained on large-scale code corpora and instruction-tuned for programming tasks. In addition, general-purpose instruction-tuned models such as *Mistral-7B-Instruct* [6] demonstrate strong reasoning capabilities despite their moderate parameter size.

Although these models achieve competitive performance on code generation and programming benchmarks, their ability to generate coherent repository-level documentation remains underexplored. In particular, repository-level documentation requires capturing architectural relationships and long-context dependencies that extend beyond individual functions or files.

D. Positioning of This Work

This study builds upon prior work on repository-level documentation generation by systematically evaluating compact open-source LLMs on the *CodeWikiBench* benchmark [3]. Unlike previous approaches that rely on proprietary large-scale models, all experiments in this work are conducted in a fully offline setting with 4-bit quantized models, enabling reproducibility and deployment in restricted HPC environments.

By comparing multiple 7B-scale open-source models on large-scale repositories, this study investigates the trade-offs between model capacity, computational cost, and documentation quality in repository-level documentation generation.

II. METHODOLOGY AND EXPERIMENTAL SETUP

A. Overall Framework

Figure 1 summarizes the end-to-end workflow of our system: we parse structured repository documentation from *CodeWikiBench*, construct a unified prompt, run fully offline inference using compact 7B open-source LLMs to generate Markdown documentation, and finally compute evaluation metrics (ROUGE-L and semantic coverage).

B. Dataset: *CodeWikiBench*

We conduct our experiments on *CodeWikiBench* [3], [8], a repository-level benchmark designed for evaluating holistic documentation generation systems. We use the publicly released Hugging Face version of the dataset [8], which contains 22 large-scale open-source repositories spanning diverse domains such as web frameworks, infrastructure systems, and developer tools. Each repository corresponds to a fixed GitHub commit snapshot to ensure reproducibility.

For each repository, the dataset provides repository metadata (repository URL and commit ID), the original documentation tree (*docs_tree*), parsed repository-level documentation (*structured_docs*), and a hierarchical rubric specification (*rubrics*).

The benchmark covers 22 repositories including *Chart.js*, *FluentValidation*, *OpenHands*, *electron*, *git-credential-manager*, *graphrag*, *json*, *libsql*, *logstash*, *marktext*, *material-components-android*, *mermaid*, *ml-agents*, *puppeteer*, *qmk_firmware*, *rasa*, *storybook*, *sumatrapdf*, *svelte*, *trino*, *wazuh*, and *x64dbg*.

Data Format. Each dataset record follows the structure below. The fields *docs_tree*, *structured_docs*, and *rubrics* are stored as serialized JSON strings and must be parsed before processing.

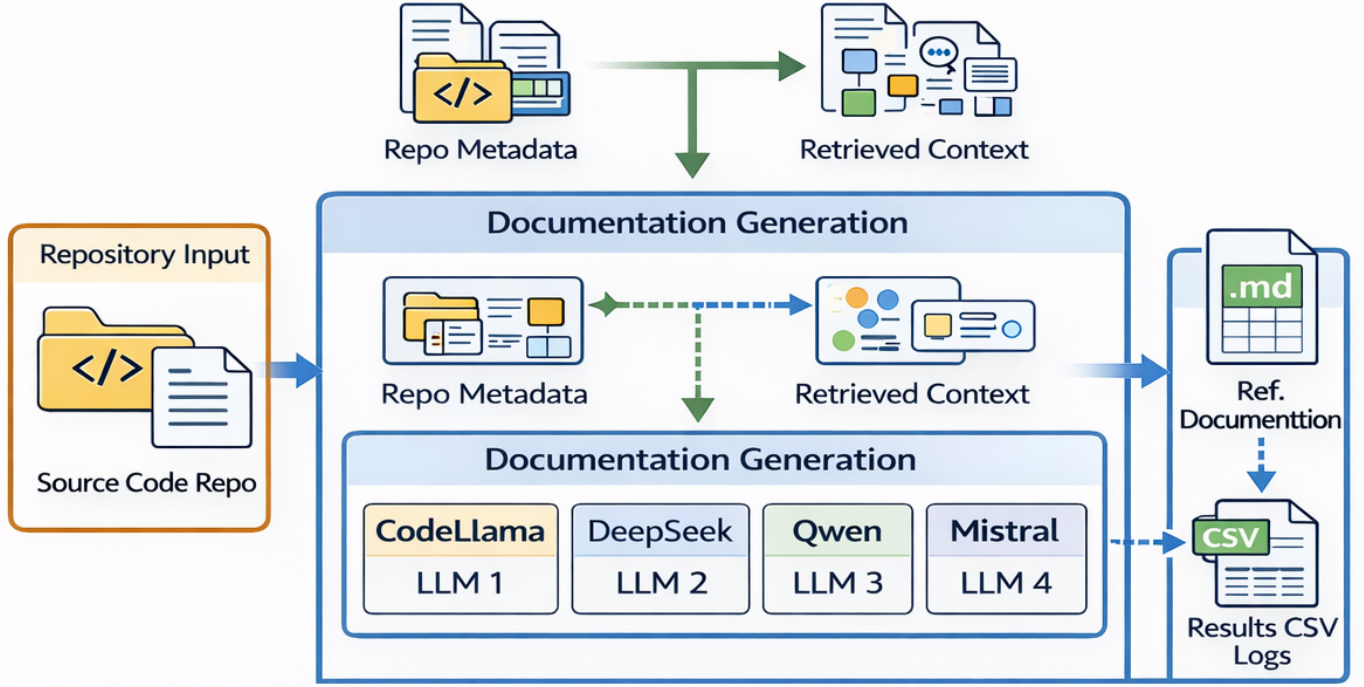


Fig. 1: Overall repository-level documentation generation and evaluation framework.

```
{
  "repo_name": str, # Name of the repository
  "repo_url": str, # GitHub repository URL
  "commit_id": str, # Fixed Git commit identifier
  "docs_tree": str, # JSON string of docs tree structure
  "structured_docs": str, # JSON string of structured documentation
  "rubrics": str # JSON string of evaluation rubrics
}
```

Usage. The dataset can be accessed via the Hugging Face datasets library:

```
from datasets import load_dataset
import json

dataset = load_dataset("anhnh2002/codewikibench")
repo_data = dataset["train"][0]

docs_tree = json.loads(repo_data["docs_tree"])
structured_docs = json.loads(repo_data["structured_docs"])
rubrics = json.loads(repo_data["rubrics"])
```

C. Problem Formulation

Given a repository R with structured repository representation $\mathcal{S}_R$, the goal is to generate a repository-level Markdown documentation file $\hat{D}_R$:

$$\hat{D}_R = \text{LLM}(\mathcal{S}_R, \mathcal{P}),$$

where $\mathcal{P}$ denotes a unified prompt template applied consistently across models. The representation $\mathcal{S}_R$ summarizes repository structure, including modules, configuration files, dependencies, and execution-related artifacts extracted from the codebase.

This task requires long-range reasoning and structural synthesis and differs substantially from short-form summarization settings that operate at the function or class level. Therefore, we evaluate the generated documentation using structure-aware evaluation metrics derived from repository documentation units.

D. Architecture-Aware Documentation Pipeline

To improve repository-level reasoning, we extend the baseline documentation generation pipeline with several architecture-aware components that explicitly capture structural relationships within the repository.

a) Repository Graph Construction: We first construct a repository dependency graph $G_R = (V, E)$, where nodes represent files or modules and edges represent structural relationships such as imports, build dependencies, or configuration references. This graph captures cross-module interactions that are often difficult to infer from isolated file summaries.

b) *Hierarchical Decomposition*: Large repositories are decomposed into a hierarchical structure consisting of directories, modules, and files. This hierarchical decomposition allows the system to process the repository at multiple abstraction levels and organize information before documentation synthesis.

c) *Architecture Intermediate Representation (IR)*: Using the dependency graph and hierarchical structure, we construct an *architecture intermediate representation* (IR) summarizing key components, their responsibilities, and their relationships. The IR acts as a condensed structural representation of the repository and serves as the primary input for documentation generation.

d) *Section-Aware Retrieval*: Instead of generating the documentation in a single pass, the system performs section-aware retrieval. For each documentation section (e.g., *Architecture*, *Execution Flow*, or *Configuration*), the system retrieves the most relevant components from the IR and repository graph. This targeted retrieval helps the model focus on relevant structural information when generating each documentation section.

e) *Hierarchical Summarization*: Finally, documentation synthesis is performed using a hierarchical summarization strategy. Lower-level summaries (e.g., module descriptions) are first generated and then aggregated into higher-level sections using a map–reduce style process. This hierarchical summarization improves coherence and enables compact models to reason over large repositories without requiring the entire repository to be processed simultaneously.

E. Generated Documentation Output (.md)

For each repository, the system generates a Markdown documentation file named after the repository (e.g., `Chart.js.md`) and stores it under a model-specific directory. Each generated document follows a stable, human-readable outline to support both qualitative inspection and structure-aware evaluation.

Concretely, the generated Markdown typically includes:

- **Overview**: a high-level description of the repository’s purpose and scope;
- **Architecture**: a summary of major components and their responsibilities, often accompanied by a high-level diagram;
- **Data Flow / Execution Flow**: descriptions of typical runtime or build processes;
- **Configuration & Dependencies**: key configuration files and dependency information;
- **How to Run / Key Scripts**: common commands used to build, run, or test the system.

We treat the generated .md files as *first-class artifacts* of the benchmark. They constitute the primary evaluation objects and enable manual error analysis (e.g., missing architectural components, incomplete module coverage, or hallucinated commands).

F. Prompt Construction

We construct a unified prompt containing three components: (i) repository metadata (repository name and commit identifier), (ii) the parsed repository representation S_R , and (iii) explicit instructions requesting a holistic repository-level documentation summary in Markdown format.

The prompt format is identical across models to ensure controlled comparison. The instruction explicitly requires a structured Markdown document with well-defined sections and headings, improving both readability and downstream parsing for evaluation.

G. Evaluated Models

We evaluate four instruction-tuned open-source models: DeepSeek-Coder-6.7B-Instruct [5], Qwen2.5-Coder-7B-Instruct [7], Mistral-7B-Instruct-v0.3 [6], and CodeLlama-7B-Instruct [4].

All experiments are executed fully offline without reliance on proprietary APIs, ensuring reproducible benchmarking in restricted or air-gapped environments.

H. Evaluation Metrics

Repository-level documentation generation does not admit a single canonical wording. Different documents may describe the same architectural structure using different phrasing or organization. Therefore, we evaluate documentation quality using structure-aware metrics derived from the benchmark evaluation rubric.

a) *Coverage Score*: Coverage score measures how much of the reference documentation structure is reflected in the generated Markdown documentation.

b) *Hierarchy Alignment*: This metric evaluates whether the generated documentation preserves the hierarchical organization of repository components.

c) *Structural Fidelity*: Structural fidelity measures how accurately the generated documentation reflects the architectural structure of the repository.

d) *Factual Grounding*: Factual grounding evaluates whether the generated documentation is supported by repository artifacts rather than hallucinated content.

e) *Actionability*: This metric measures whether the documentation provides practical guidance for users, such as installation instructions, configuration details, or execution commands.

f) *Coherence*: Coherence evaluates the readability and logical flow of the generated documentation.

I. Result Logs and Metric Interpretation (results/*.csv)

Alongside generated Markdown files, the system produces per-model CSV logs stored under `outputs/results/`. Each row corresponds to a single repository evaluated by a single model and links the generated documentation artifact with the computed metrics.

Each CSV row contains fields including the repository identifier, model name, generated document path, and the set of evaluation metrics used in our analysis.

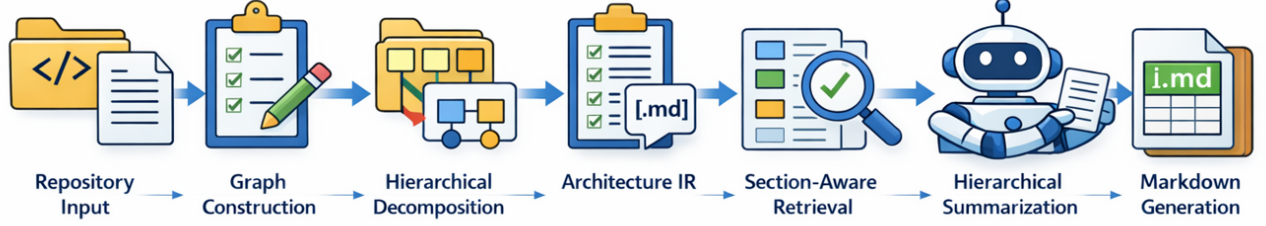


Fig. 2: Architecture-aware repository documentation generation pipeline.

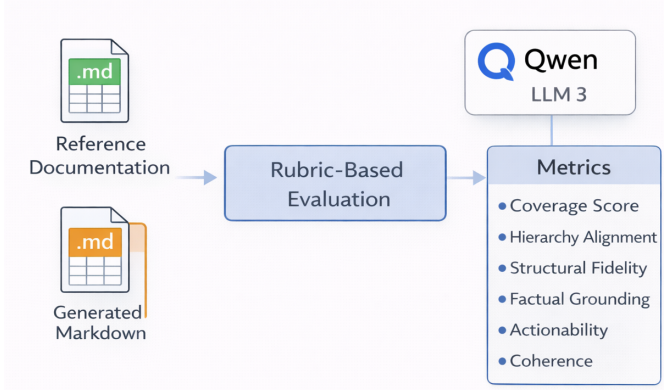


Fig. 3: Evaluation workflow for repository-level documentation generation.

These logs serve as the primary artifact used to aggregate results across repositories and compute model-level averages.

J. Evaluation Workflow

The evaluation workflow compares generated Markdown documentation with reference documentation extracted from the benchmark dataset. A rubric-based evaluation procedure then computes structure-aware metrics such as coverage score, hierarchy alignment, structural fidelity, factual grounding, actionability, and coherence.

K. Reproducibility

All experiments are conducted in a fully offline setting. Models are loaded locally and evaluated without external API calls. Generated Markdown files and evaluation logs are stored deterministically, ensuring transparent and reproducible benchmarking.

III. RESULTS AND DISCUSSION

A. Experimental Setup

We evaluate repository-level documentation generation on the CodeWikiBench dataset. For each repository, the system produces a Markdown documentation artifact stored under `outputs/docs/<MODEL>/`. Evaluation metrics are computed using the benchmark evaluation pipeline described in Section ??.

To analyze the effectiveness of the proposed architecture-aware documentation pipeline, we compare two configurations:

- **Baseline pipeline (V1):** direct documentation generation using the structured repository representation.
- **Architecture-aware pipeline (V2):** the proposed system that incorporates repository graph construction, hierarchical decomposition, architecture intermediate representations, section-aware retrieval, and hierarchical summarization.

We evaluate four instruction-tuned open-source models: CodeLlama-7B-Instruct, DeepSeek-Coder-6.7B-Instruct, Mistral-7B-Instruct-v0.3, and Qwen2.5-Coder-7B-Instruct. All experiments are executed fully offline using locally hosted models.

B. Overall Performance

Table I compares documentation generation quality for the four evaluated models under both pipeline configurations.

The results show that the proposed architecture-aware pipeline consistently improves documentation generation quality across all four evaluated models. In particular, coverage and structural fidelity both increase under the V2 pipeline, indicating that explicit modeling of repository structure helps compact language models better capture architectural relationships within large codebases.

Quality-related metrics such as factual grounding and actionability also improve in the V2 setting. This suggests that section-aware retrieval helps the model identify relevant repository components, while hierarchical summarization improves the clarity and usefulness of the generated documentation.

C. Ablation Study

To better understand the contribution of each component in the proposed pipeline, we conduct an ablation study that progressively introduces each architectural module.

The ablation results show that each component contributes incrementally to improving repository-level documentation coverage. The largest improvement occurs after introducing section-aware retrieval, highlighting the importance of targeted structural context during documentation generation.

TABLE I: Comparison of documentation generation quality for four models under the baseline pipeline (V1) and the proposed architecture-aware pipeline (V2).

Model	Baseline Pipeline (V1)					Architecture-aware Pipeline (V2)				
	Coverage	Structural	Factual	Action	Coherence	Coverage	Structural	Factual	Action	Coherence
CodeLlama-7B	0.55	0.79	0.77	0.74	0.82	0.63	0.84	0.82	0.81	0.85
DeepSeek-6.7B	0.54	0.78	0.76	0.73	0.81	0.60	0.82	0.80	0.78	0.84
Mistral-7B-v0.3	0.55	0.80	0.78	0.75	0.83	0.59	0.83	0.81	0.79	0.85
Qwen2.5-7B	0.56	0.80	0.78	0.76	0.83	0.62	0.84	0.82	0.80	0.86

D. Qualitative Analysis

In addition to quantitative evaluation metrics, the generated Markdown documents enable qualitative inspection of repository-level documentation.

Most generated documents follow a structured format that includes sections such as *Overview*, *Architecture*, *Execution Flow*, *Configuration and Dependencies*, and *Usage Instructions*. This structure improves readability and aligns with common software documentation practices.

The architecture-aware pipeline encourages models to synthesize higher-level repository structure rather than merely summarizing individual files. Generated documents frequently identify major modules, describe repository workflows, and provide commands required to build or execute the system.

Nevertheless, several limitations remain. Architecture diagrams may still be relatively shallow, execution flows can be overly generic, and configuration sections sometimes enumerate files without fully explaining their functional roles.

E. Discussion

Our experiments highlight the importance of structural context in repository-level documentation generation. The architecture-aware pipeline introduces explicit structural representations through repository graphs, hierarchical decomposition, and section-aware retrieval. These representations provide strong contextual signals that enable compact language models to reason about repository components and their relationships.

Overall, the experimental results suggest that improvements in documentation generation depend not only on model scale but also on the quality of the structural representations provided to the model. By exposing architectural relationships explicitly, the proposed pipeline enables compact open-source models to produce more complete and coherent documentation for large software repositories.

IV. CONCLUSION AND FUTURE WORK

This work investigates repository-level documentation generation using compact open-source large language models in a fully offline setting. We propose an architecture-aware documentation pipeline that explicitly models repository structure through repository graph construction, hierarchical decomposition, architecture intermediate representations, section-aware retrieval, and hierarchical summarization.

We evaluate the proposed system on the CodeWikiBench dataset using four instruction-tuned open-source models:

CodeLlama-7B-Instruct, DeepSeek-Coder-6.7B-Instruct, Mistral-7B-Instruct-v0.3, and Qwen2.5-Coder-7B-Instruct. Experimental results show that the architecture-aware pipeline consistently improves documentation quality compared with the baseline generation pipeline across multiple evaluation metrics. In particular, structural metrics such as coverage score and hierarchy alignment improve under the proposed V2 pipeline, indicating that explicit structural representations enable compact models to better capture architectural relationships within large software repositories.

Beyond improving structural coverage, the proposed pipeline also enhances qualitative aspects of the generated documentation, including factual grounding, actionability, and overall coherence. These improvements suggest that combining repository-level structural signals with hierarchical summarization helps compact language models synthesize more useful and readable documentation artifacts.

More broadly, our findings highlight that improving repository-level documentation generation does not rely solely on scaling model parameters. Instead, providing richer architectural representations of software repositories can significantly improve the reasoning ability of compact open-source models when generating holistic documentation.

A. Future Work

Several directions remain for future research.

First, richer architectural representations could be explored to better capture complex relationships within large repositories. For example, incorporating build graphs, runtime traces, or dependency analysis may provide additional structural signals for documentation generation.

Second, future work may investigate stronger semantic evaluation methods that go beyond structure-aware coverage metrics. Embedding-based similarity measures or calibrated evaluation using larger language models may provide more robust assessments of documentation quality.

Third, hierarchical or staged generation pipelines could further improve long-context reasoning for repository-level documentation. For instance, coarse-to-fine generation strategies may help models synthesize architectural information more effectively.

Finally, scaling experiments with larger open-source models may help clarify the interaction between model capacity and architectural representations in repository-level documentation generation.

REFERENCES

- [1] Sonia Haiduc, Jairo Aponte, Laura Moreno, and Andrian Marcus. On the use of automated text summarization techniques for summarizing source code. In *WCRE*, 2010.
- [2] Giriprasad Sridhara, Emily Hill, Divya Muppaneni, Lori Pollock, and K Vijay-Shanker. Towards automatically generating summary comments for java methods. In *ASE*, 2010.
- [3] Anh N. Hoang et al. Codewiki: Evaluating ai’s ability to generate holistic documentation for large-scale codebases. *arXiv preprint arXiv:2510.24428*, 2025.
- [4] Baptiste Rozière et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- [5] DeepSeek-AI. Deepseek-coder: When the large language model meets programming. *arXiv preprint*, 2024.
- [6] Albert Q. Jiang et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [7] Qwen Team. Qwen2.5 technical report. *arXiv preprint*, 2024.
- [8] Codewikibench dataset. <https://huggingface.co/datasets/anhnh2002/codewikibench>, 2025.
- [9] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.